EXP. No. 6	BASH SCRIPT USING BASH VARIABLE
DATE:	

AIM:

To write a Bash shell script to calculate an employee's basic salary details and arithmetic equation

ALGORITHM:

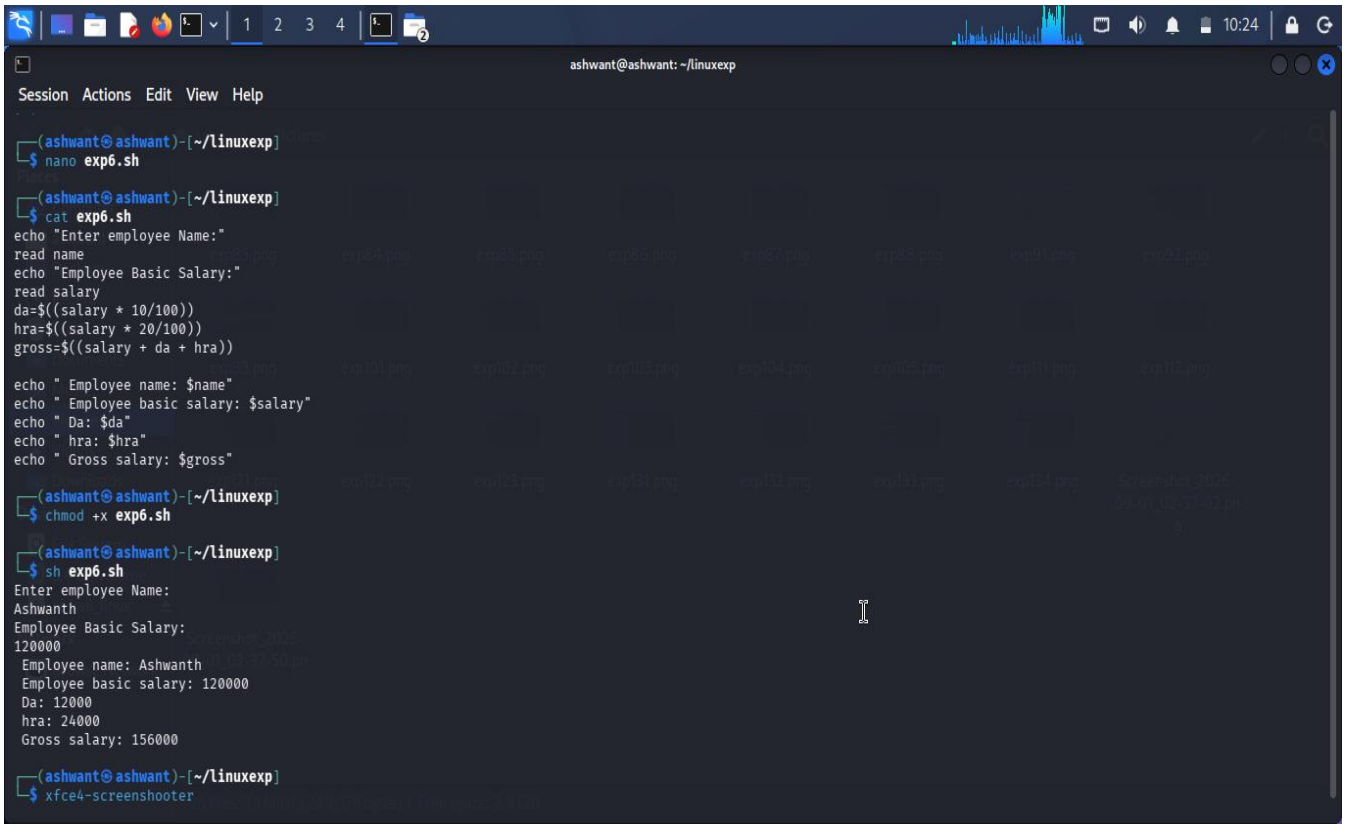
1. Start the program.
2. Read the employee name.
3. Read the basic salary.
4. Calculate HRA as 20% of the basic salary.
5. Calculate DA as 10% of the basic salary.
6. Calculate gross salary = Basic salary + HRA + DA.
7. Display the employee details and salary.

COMMANDS:

1. To create shell file:
   $ nano emp.sh
2. #Scripting

```
echo "Enter employee name:"
read name
echo "Employee Basic Salary:"
read salary
da=$((salary * 10 / 100))
hra=$((salary * 20 / 100))
gross=$((salary + da + hra))
echo "Employee name : $name"
echo "Employee basic salary : $salary"
echo "DA : $da"
echo "HRA : $hra"
echo "Gross salary : $gross"
```

3. To give permission to execute:
   $ chmod +x emp.sh
4. To run the shell:
   $ ./emp.sh


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
(ashwant㉿ashwant)-[~/linuxexp]
$ nano exp6.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ cat exp6.sh
echo "Enter employee Name:"
read name
echo "Employee Basic Salary:"
read salary
da=$((salary * 10/100))
hra=$((salary * 20/100))
gross=$((salary + da + hra))
echo " Employee name: $name"
echo " Employee basic salary: $salary"
echo " Da: $da"
echo " hra: $hra"
echo " Gross salary: $gross"
(ashwant㉿ashwant)-[~/linuxexp]
$ chmod +x exp6.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ sh exp6.sh
Enter employee Name:
Ashwanth
Employee Basic Salary:
120000
Employee name: Ashwanth
Employee basic salary: 120000
Da: 12000
hra: 24000
Gross salary: 156000
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter

PROGRAM AND EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, using Linux commands, employee gross salary has been found successfully.

EXP. No. 7	BASH SCRIPT USING CONDITIONAL STATEMENT
DATE:	

AIM:

To write a Bash shell script to validate the therapy session details such as Patient ID, Patient Name, Age, Gender, Therapist Name, Session Date, Session Time, Username and Password, and check whether valid.

ALGORITHM:

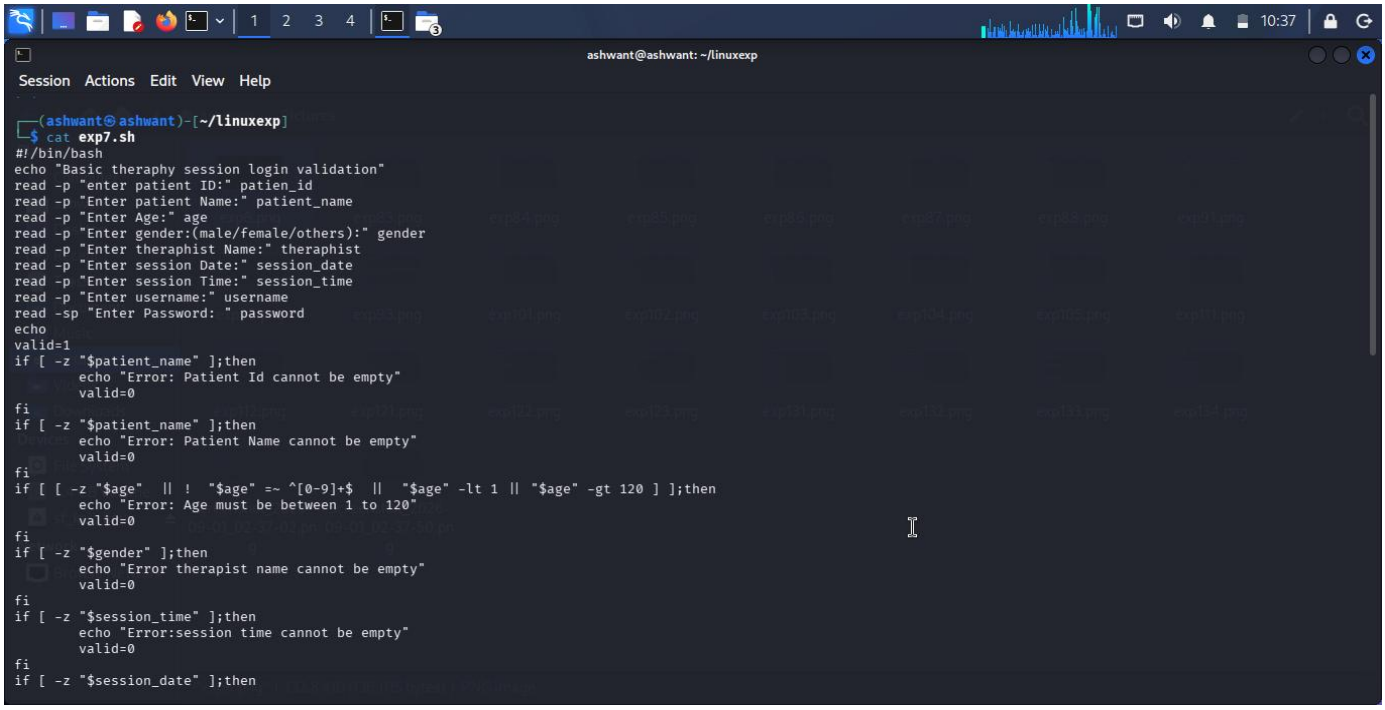
1. Read Patient ID, Patient Name, Age, Gender, Therapist Name, Session Time, Date, Username & Password.
2. Check whether each required field is empty.
3. Check whether Age is a number between 1 to 120.
4. If any field is invalid, display the corresponding error message.

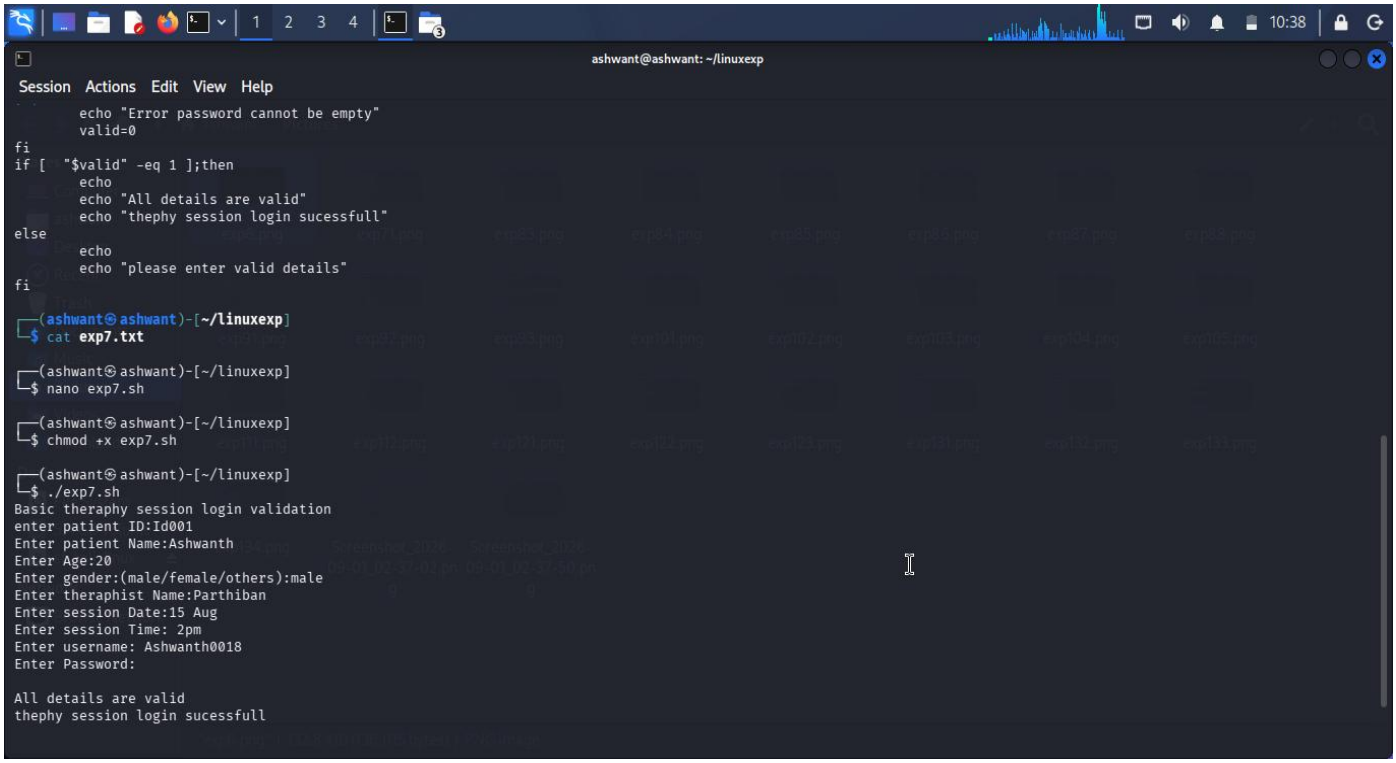
COMMANDS:

1.To create script:

$nano emp7.sh

```
#!/bin/bash
echo "Basic Therapy Session Login Validation"
read -p "Enter Patient ID: " patient_id
read -p "Enter Patient Name: " patient_name
read -p "Enter Age: " age
read -p "Enter Gender (Male/Female/Other): " gender
read -p "Enter Therapist Name: " therapist
read -p "Enter Session Date: " session_date
read -p "Enter Session Time: " session_time
read -p "Enter Username: " username
read -sp "Enter Password: " password
echo
valid=1
if [ -z "$patient_id" ]; then
echo "Error: Patient ID cannot be empty"
valid=0
fi
if [ -z "$patient_name" ]; then
echo "Error: Patient name cannot be empty"
valid=0
fi
if ! [[ "$age" =~ ^[0-9]+$ ]] || [ "$age" -lt 1 ] || [ "$age" -gt 120 ]; then
echo "Error: Age must be between 1 and 120"
valid=0
fi
```


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
┌──(ashwant㉿ashwant)-[~/linuxexp]
└─$ cat exp7.sh
#!/bin/bash
echo "Basic theraphy session login validation"
read -p "enter patient ID:" patien_id
read -p "Enter patient Name:" patient_name
read -p "Enter Age:" age
read -p "Enter gender:(male/female/others):" gender
read -p "Enter theraphist Name:" theraphist
read -p "Enter session Date:" session_date
read -p "Enter session Time:" session_time
read -p "Enter username:" username
read -sp "Enter Password: " password
echo
valid=1
if [-z "$patient_name"];then
echo "Error: Patient Id cannot be empty"
valid=0
fi
if [-z "$patient_name"];then
echo "Error: Patient Name cannot be empty"
valid=0
fi
if [[-z "$age" || ! "$age" =~ ^[0-9]+$ || "$age" -lt 1 || "$age" -gt 120]];then
echo "Error: Age must be between 1 to 120"
valid=0
fi
if [-z "$gender"];then
echo "Error therapist name cannot be empty"
valid=0
fi
if [-z "$session_time"];then
echo "Error:session time cannot be empty"
valid=0
fi
if [-z "$session_date"];then


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
echo "Error password cannot be empty"
valid=0
fi
if ["$valid" -eq 1];then
echo
echo "All details are valid"
echo "thephy session login sucessfull"
else
echo
echo "please enter valid details"
fi
┌──(ashwant㉿ashwant)-[~/linuxexp]
└─$ cat exp7.txt
┌──(ashwant㉿ashwant)-[~/linuxexp]
└─$ nano exp7.sh
┌──(ashwant㉿ashwant)-[~/linuxexp]
└─$ chmod +x exp7.sh
┌──(ashwant㉿ashwant)-[~/linuxexp]
└─$ ./exp7.sh
Basic theraphy session login validation
enter patient ID:Id001
Enter patient Name:Ashwanth
Enter Age:20
Enter gender:(male/female/others):male
Enter theraphist Name:Parthiban
Enter session Date:15 Aug
Enter session Time: 2pm
Enter username: Ashwanth0018
Enter Password:
All details are valid
thephy session login sucessfull

```
if [ -z "$gender" ]; then
echo "Error: Gender cannot be empty"
valid=0
fi
if [ -z "$therapist" ]; then
echo "Error: Therapist Name cannot be empty"
valid=0
fi
if [ -z "$session_time" ]; then
echo "Error: Session Time cannot be empty"
valid=0
fi
if [ -z "$session_date" ]; then
echo "Error: Session Date cannot be empty"
valid=0
fi
if [ -z "$username" ]; then
echo "Error: Username cannot be empty"
valid=0
fi
if [ -z "$password" ]; then
echo "Error: Password cannot be empty"
valid=0
fi
if [ "$valid" -eq 1 ]; then
echo "All details are valid"
echo "Basic Therapy Session Login Successful!"
else
echo "Please enter valid details."
fi
```

2.To give execute permission:

$ chmod +x emp7.sh

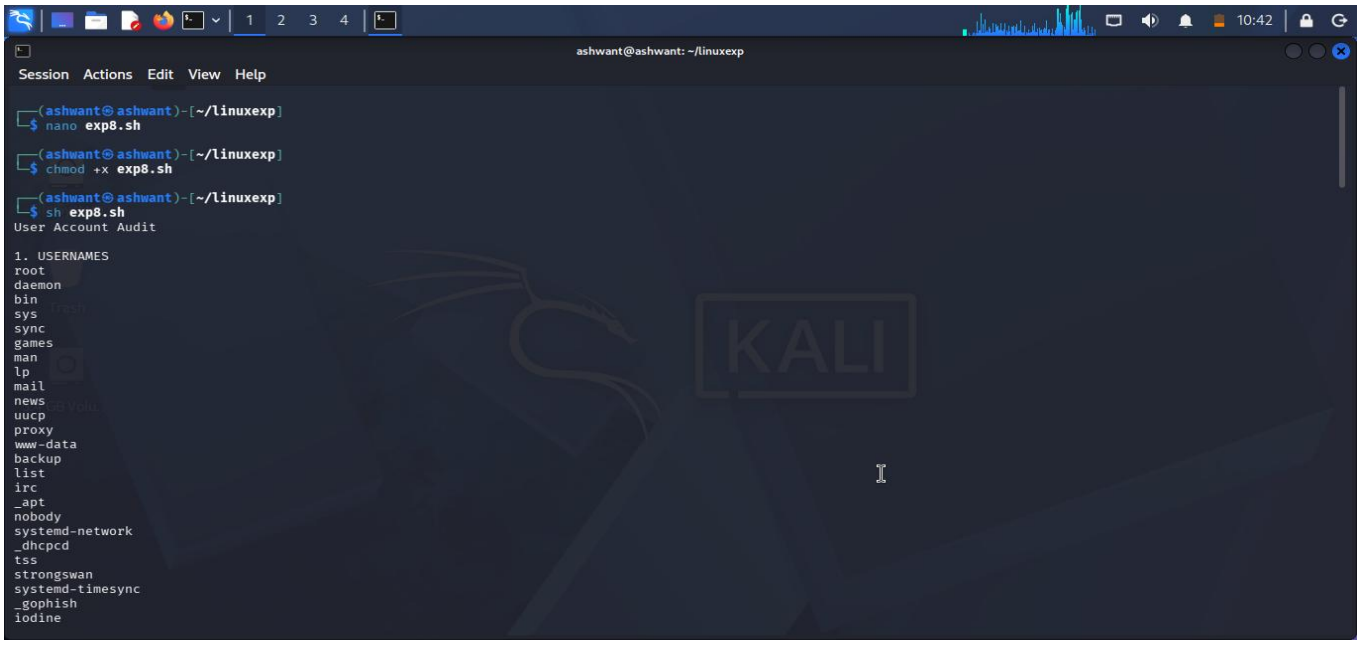
3.To run the script:

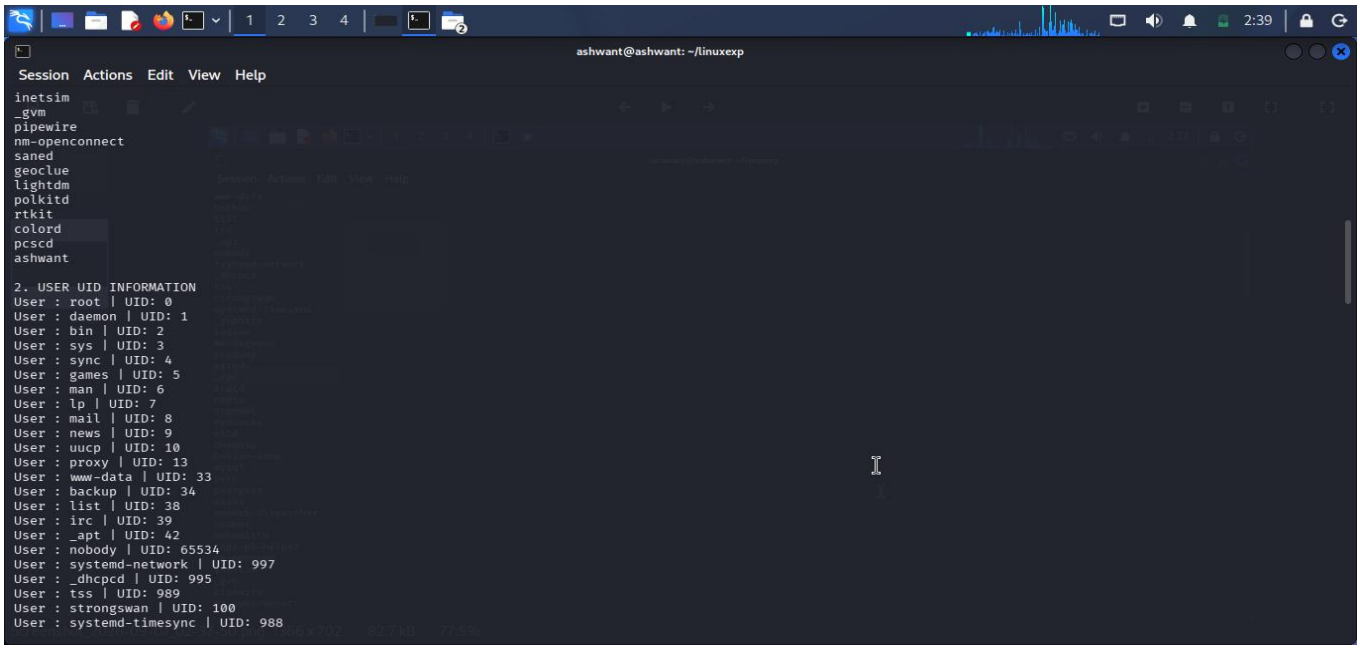
$ ./emp7.sh

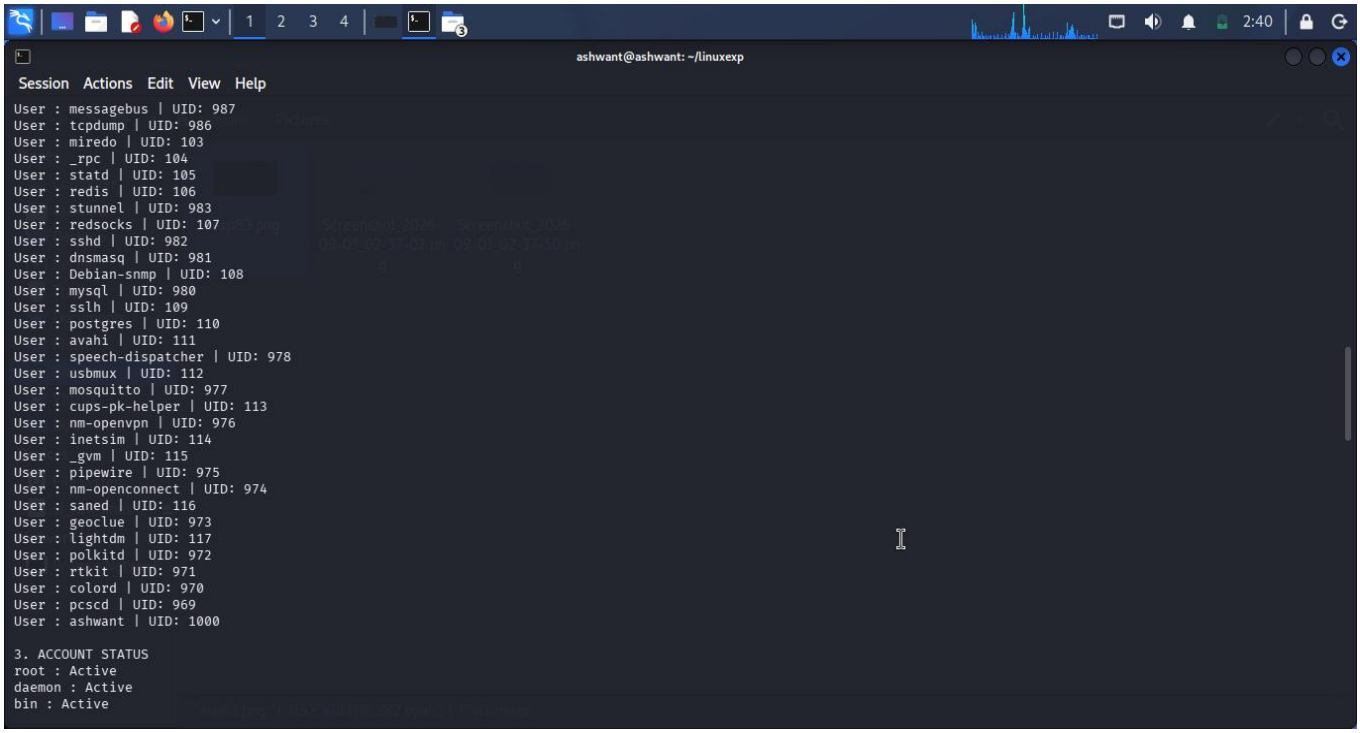
PROGRAM AND EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the login validation script was executed successfully and the therapy session details were validated.


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
(ashwant㉿ashwant)-[~/linuxexp]
$ nano exp8.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ chmod +x exp8.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ sh exp8.sh
User Account Audit
1. USERNAMES
root
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
list
irc
_apt
nobody
systemd-network
_dhcpcd
tss
strongswan
systemd-timesync
_gophish
iodine


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
inetsim
_gvm
pipewire
nm-openconnect
saned
geoclue
lightdm
polkitd
rtkit
colord
pcscd
ashwant
2. USER UID INFORMATION
User : root | UID: 0
User : daemon | UID: 1
User : bin | UID: 2
User : sys | UID: 3
User : sync | UID: 4
User : games | UID: 5
User : man | UID: 6
User : lp | UID: 7
User : mail | UID: 8
User : news | UID: 9
User : uucp | UID: 10
User : proxy | UID: 13
User : www-data | UID: 33
User : backup | UID: 34
User : list | UID: 38
User : irc | UID: 39
User : _apt | UID: 42
User : nobody | UID: 65534
User : systemd-network | UID: 997
User : _dhcpcd | UID: 995
User : tss | UID: 989
User : strongswan | UID: 100
User : systemd-timesync | UID: 988


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
User : messagebus | UID: 987
User : tcpdump | UID: 986
User : miredo | UID: 103
User : _rpc | UID: 104
User : statd | UID: 105
User : redis | UID: 106
User : stunnel | UID: 983
User : redsocks | UID: 107
User : sshd | UID: 982
User : dnsmasq | UID: 981
User : Debian-snmp | UID: 108
User : mysql | UID: 980
User : sslh | UID: 109
User : postgres | UID: 110
User : avahi | UID: 111
User : speech-dispatcher | UID: 978
User : usbmux | UID: 112
User : mosquitto | UID: 977
User : cups-pk-helper | UID: 113
User : nm-openvpn | UID: 976
User : inetsim | UID: 114
User : _gvm | UID: 115
User : pipewire | UID: 975
User : nm-openconnect | UID: 974
User : saned | UID: 116
User : geoclue | UID: 973
User : lightdm | UID: 117
User : polkitd | UID: 972
User : rtkit | UID: 971
User : colord | UID: 970
User : pcscd | UID: 969
User : ashwant | UID: 1000
3. ACCOUNT STATUS
root : Active
daemon : Active
bin : Active

EXP. No. 8	BASH SCRIPT USING LOOPS
DATE:	

AIM:

To write a Bash shell script using a loop for user account management..

ALGORITHM:

1. Display a menu for user account management.
2. Read the user account operation.
3. Use a loop to repeatedly perform the selected operation.
4. Create, delete, or display user accounts using appropriate Linux commands.
5. Continue until the user selects the quit option.

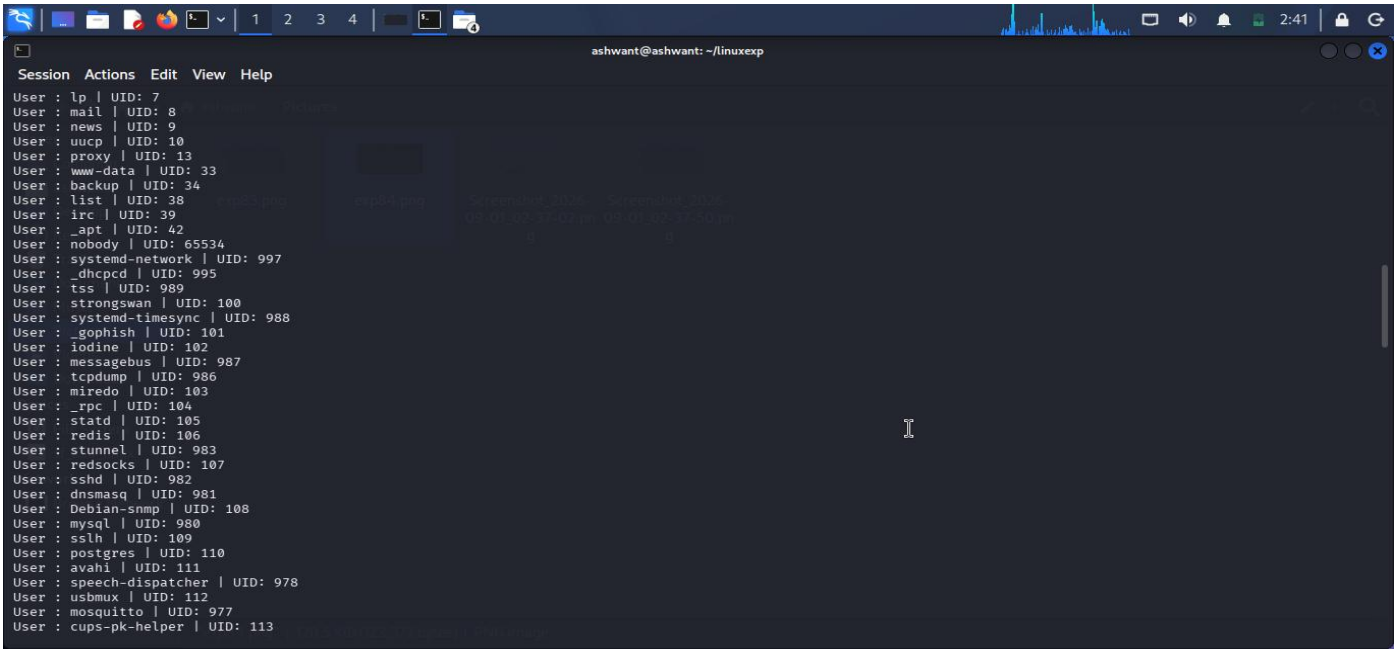
COMMANDS:

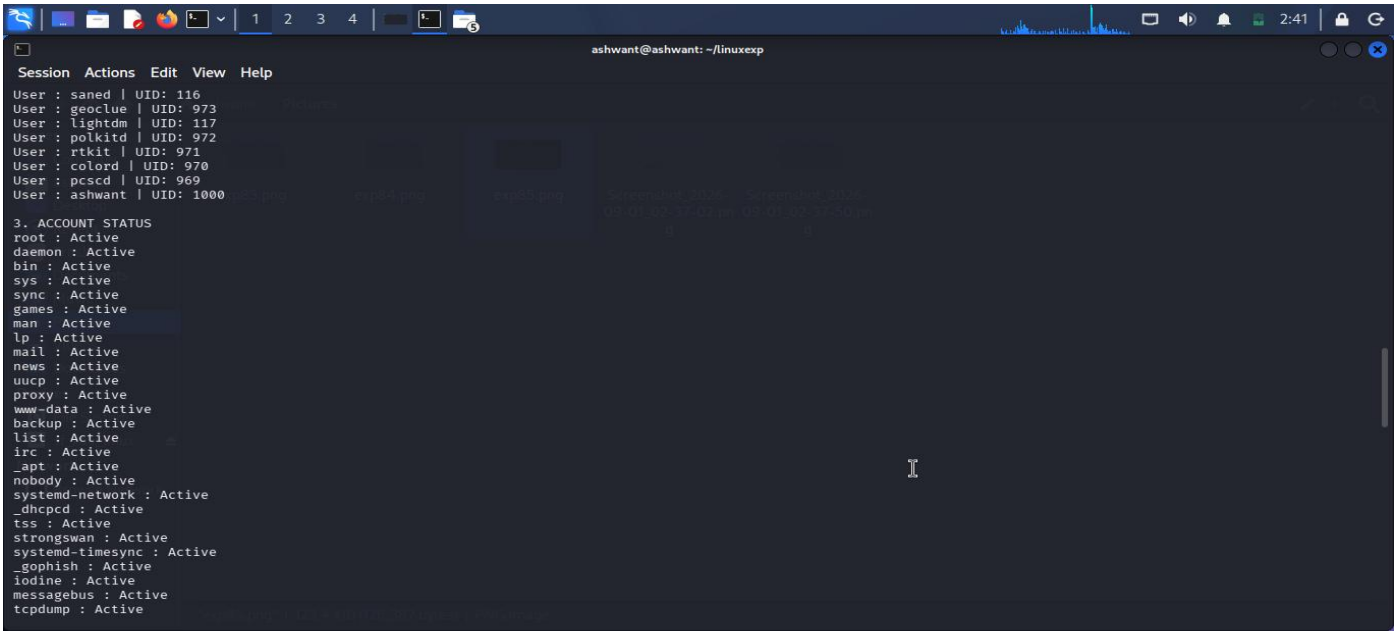
1.To create script:

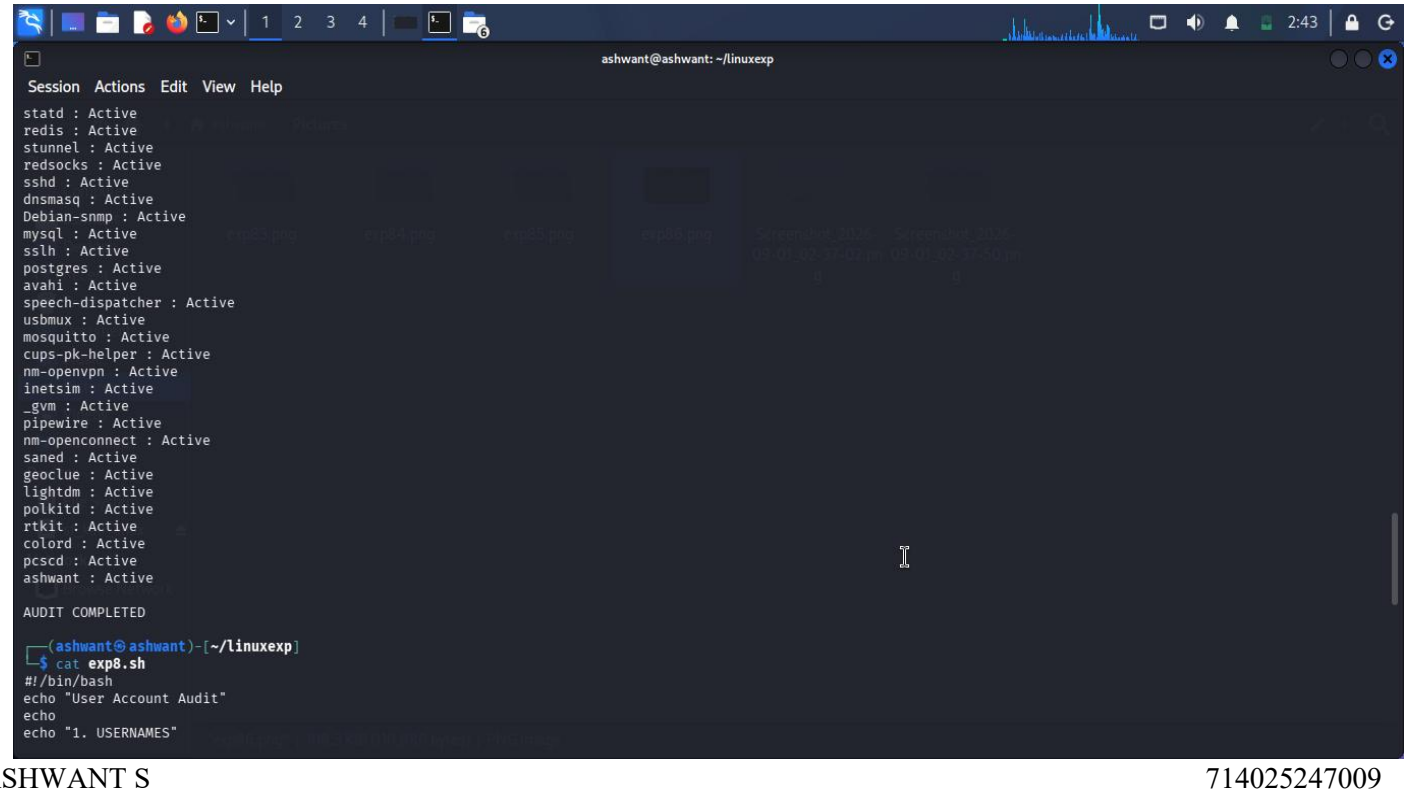
$ nano emp8.sh

#Scripting:

```
#!/bin/bash
echo "User Account Audit"
echo
echo "1. Username"
for user in $(cut -d: -f1 /etc/passwd)
do
echo "$user"
done
echo
echo "2. USER UID INFORMATION"
cat /etc/passwd | while IFS=: read -r username password uid gid fullname home shell
do
echo "User: $username | UID: $uid"
done
echo
echo "3. Account Status"
users=$(cut -d: -f1 /etc/passwd)
i=1
total=$(echo "$users" | wc -l)
until [ $i -gt $total ]
do
username=$(echo "$users" | sed -n "${i}p")
if passwd -S "$username" 2>/dev/null | grep -q " L "
then
echo "$username : Locked"
```


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
User : lp | UID: 7
User : mail | UID: 8
User : news | UID: 9
User : uucp | UID: 10
User : proxy | UID: 13
User : www-data | UID: 33
User : backup | UID: 34
User : list | UID: 38
User : irc | UID: 39
User : _apt | UID: 42
User : nobody | UID: 65534
User : systemd-network | UID: 997
User : _dhcpcd | UID: 995
User : tss | UID: 989
User : strongswan | UID: 100
User : systemd-timesync | UID: 988
User : _gophish | UID: 101
User : iodine | UID: 102
User : messagebus | UID: 987
User : tcpdump | UID: 986
User : miredo | UID: 103
User : _rpc | UID: 104
User : statd | UID: 105
User : redis | UID: 106
User : stunnel | UID: 983
User : redsocks | UID: 107
User : sshd | UID: 982
User : dnsmasq | UID: 981
User : Debian-snmp | UID: 108
User : mysql | UID: 980
User : sslh | UID: 109
User : postgres | UID: 110
User : avahi | UID: 111
User : speech-dispatcher | UID: 978
User : usbmux | UID: 112
User : mosquitto | UID: 977
User : cups-pk-helper | UID: 113


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
User : saned | UID: 116
User : geoclue | UID: 973
User : lightdm | UID: 117
User : polkitd | UID: 972
User : rtkit | UID: 971
User : colord | UID: 970
User : pcscd | UID: 969
User : ashwant | UID: 1000
3. ACCOUNT STATUS
root : Active
daemon : Active
bin : Active
sys : Active
sync : Active
games : Active
man : Active
lp : Active
mail : Active
news : Active
uucp : Active
proxy : Active
www-data : Active
backup : Active
list : Active
irc : Active
_apt : Active
nobody : Active
systemd-network : Active
_dhcpcd : Active
tss : Active
strongswan : Active
systemd-timesync : Active
_gophish : Active
iodine : Active
messagebus : Active
tcpdump : Active


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
statd : Active
redis : Active
stunnel : Active
redsocks : Active
sshd : Active
dnsmasq : Active
Debian-snmp : Active
mysql : Active
sslh : Active
postgres : Active
avahi : Active
speech-dispatcher : Active
usbmux : Active
mosquitto : Active
cups-pk-helper : Active
nm-openvpn : Active
inetsim : Active
_gvm : Active
pipewire : Active
nm-openconnect : Active
saned : Active
geoclue : Active
lightdm : Active
polkitd : Active
rtkit : Active
colord : Active
pcscd : Active
ashwant : Active
AUDIT COMPLETED
(ashwant㉿ashwant)-[~/linuxexp]
$ cat exp8.sh
#!/bin/bash
echo "User Account Audit"
echo
echo "1. USERNAMES"

```
        else
        echo "$username : Active"
        fi
        i=$((i+1))
        done
        echo
        echo "AUDIT COMPLETE"
```

2. To give execute permission:

$ chmod +x emp8.sh

3. To run the script:

$ sh emp8.sh

PROGRAM AND EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the Bash shell script for user account management using a loop has been done successfully.

EXP. No. 9	BASH SCRIPT USING FUNCTION
DATE:	

AIM:

To develop a Bash shell script using functions to automate basic administrative tasks in Linux.

ALGORITHM:

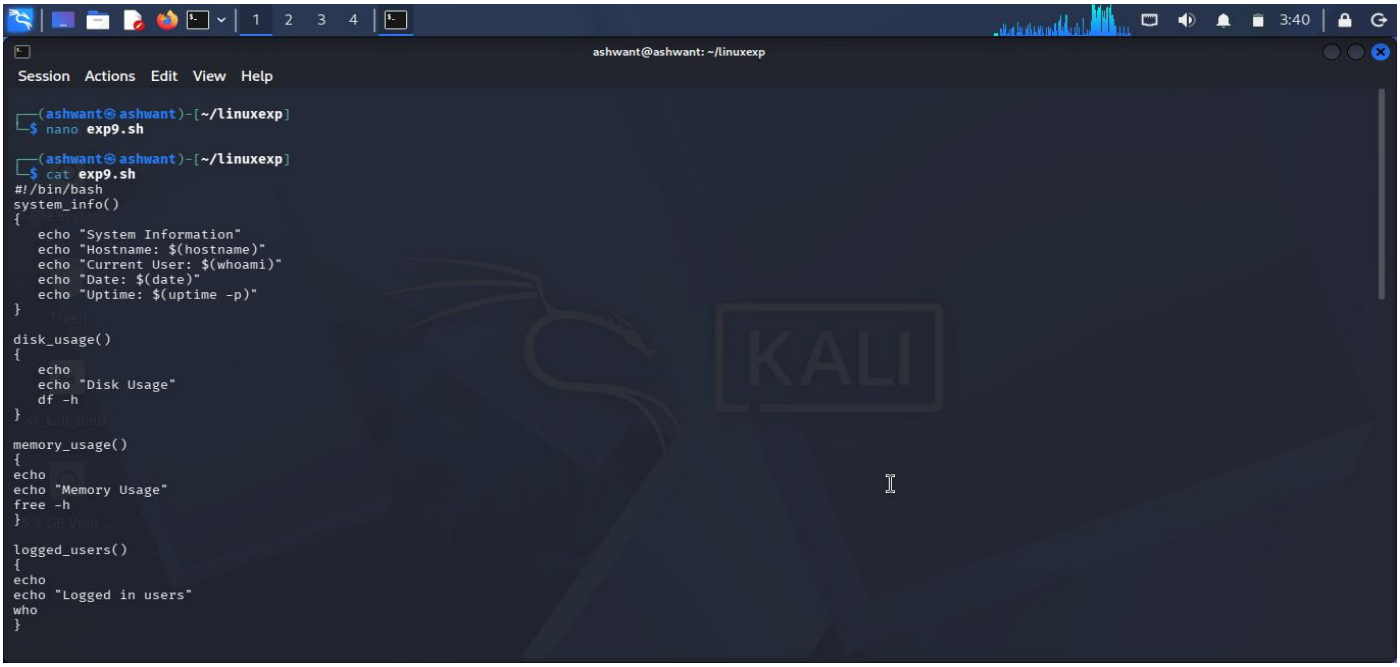
1. Write the Bash script.
2. Define functions for administrative tasks.
3. Call the required functions.
4. Display the task result.
5. Note the outputs.

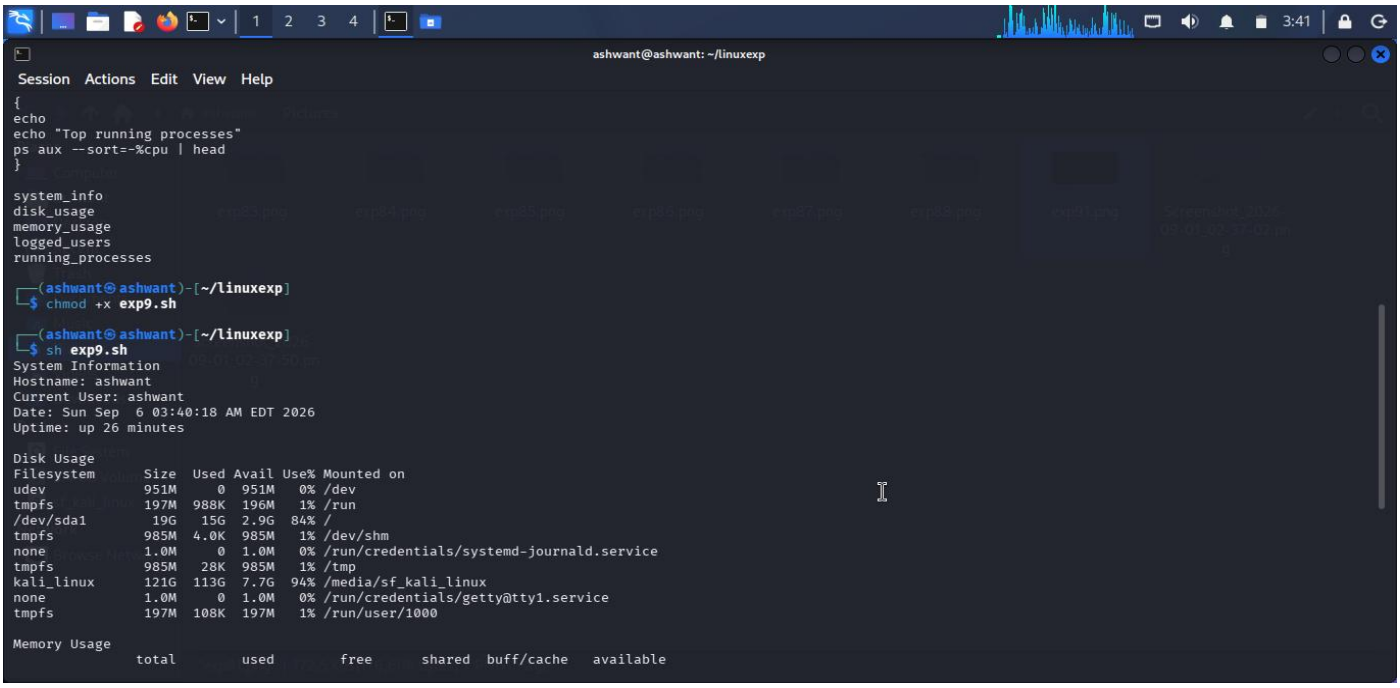
COMMANDS:

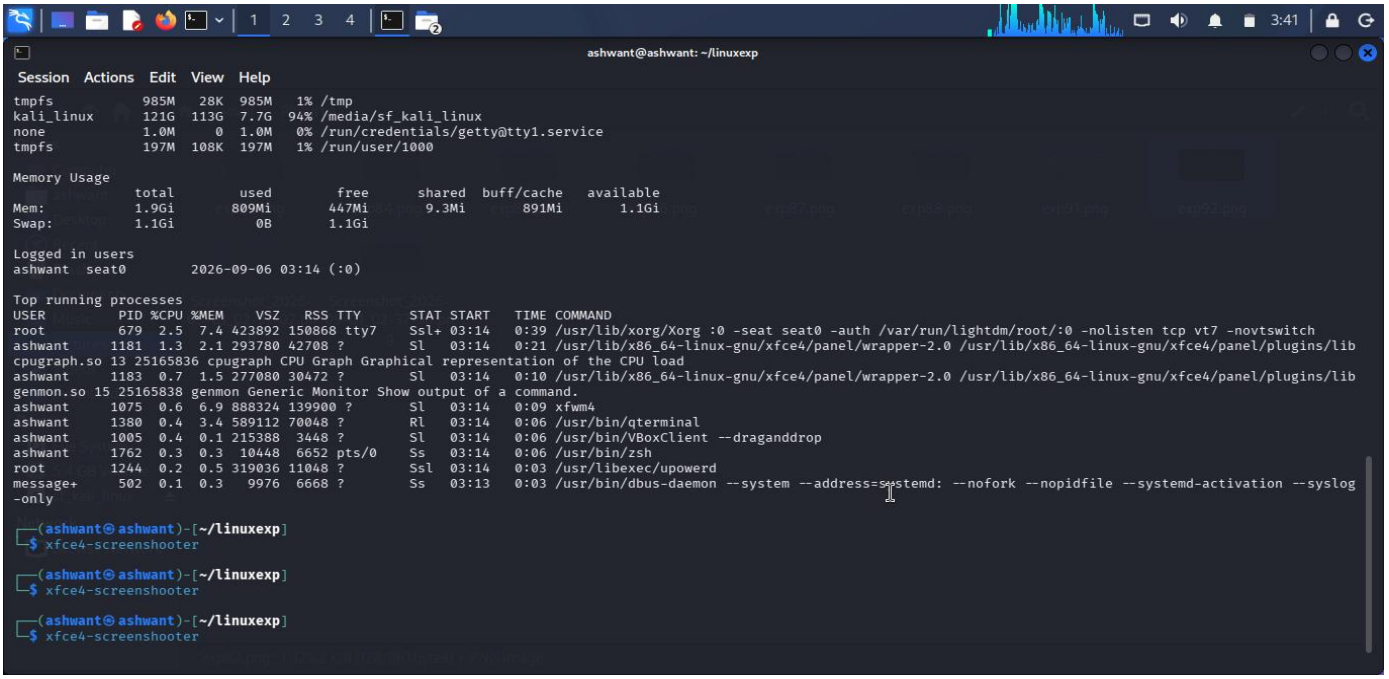
1.$ nano exp9.sh

#Scripting:

```
#!/bin/bash
dysrm-info()
{
echo "System Information"
echo "Hostname : $(hostname)"
echo "Current user : $(whoami)"
echo "Date : $(date)"
echo "Uptime : $(uptime -p)"
}
disk-usage()
{
echo "Disk Usage"
df -h
}
memory-usage()
{
echo "Memory Usage"
free -h
}
logged-users()
{
echo "Logged in users"
who
}
running-processes()
{
echo "Top running process"
```


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
(ashwant㉿ashwant)-[~/linuxexp]
$ nano exp9.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ cat exp9.sh
#!/bin/bash
system_info()
{
echo "System Information"
echo "Hostname: $(hostname)"
echo "Current User: $(whoami)"
echo "Date: $(date)"
echo "Uptime: $(uptime -p)"
}
disk_usage()
{
echo
echo "Disk Usage"
df -h
}
memory_usage()
{
echo
echo "Memory Usage"
free -h
}
logged_users()
{
echo
echo "Logged in users"
who
}


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
{
echo
echo "Top running processes"
ps aux --sort=-%cpu | head
}
system_info
disk_usage
memory_usage
logged_users
running_processes
(ashwant㉿ashwant)-[~/linuxexp]
$ chmod +x exp9.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ sh exp9.sh
System Information
Hostname: ashwant
Current User: ashwant
Date: Sun Sep 6 03:40:18 AM EDT 2026
Uptime: up 26 minutes
Disk Usage
Filesystem Size Used Avail Use% Mounted on
udev 951M 0 951M 0% /dev
tmpfs 197M 988K 196M 1% /run
/dev/sda1 19G 15G 2.9G 84% /
tmpfs 985M 4.0K 985M 1% /dev/shm
none 1.0M 0 1.0M 0% /run/credentials/systemd-journal.service
tmpfs 985M 28K 985M 1% /tmp
kali_linux 121G 113G 7.7G 94% /media/sf_kali_linux
none 1.0M 0 1.0M 0% /run/credentials/getty@tty1.service
tmpfs 197M 108K 197M 1% /run/user/1000
Memory Usage
total used free shared buff/cache available


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
tmpfs 985M 28K 985M 1% /tmp
kali_linux 121G 113G 7.7G 94% /media/sf_kali_linux
none 1.0M 0 1.0M 0% /run/credentials/getty@tty1.service
tmpfs 197M 108K 197M 1% /run/user/1000
Memory Usage
total used free shared buff/cache available
Mem: 1.9Gi 809Mi 447Mi 9.3Mi 891Mi 1.1Gi
Swap: 1.1Gi 0B 1.1Gi
Logged in users
ashwant seat0 2026-09-06 03:14 (:0)
Top running processes
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 679 2.5 7.4 423892 150868 tty7 Ssl+ 03:14 0:39 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch
ashwant 1181 1.3 2.1 293780 42708 ? Sl 03:14 0:21 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/lib
cpugraph.so 13 25165836 cpugraph CPU Graph Graphical representation of the CPU load
ashwant 1183 0.7 1.5 277080 30472 ? Sl 03:14 0:10 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/lib
genmon.so 15 25165838 genmon Generic Monitor Show output of a command.
ashwant 1075 0.6 6.9 888324 139900 ? Sl 03:14 0:09 xfwm4
ashwant 1380 0.4 3.4 589112 70048 ? Rl 03:14 0:06 /usr/bin/qterminal
ashwant 1005 0.4 0.1 215388 3448 ? Sl 03:14 0:06 /usr/bin/VBoxClient --draganddrop
ashwant 1762 0.3 0.3 10448 6652 pts/0 Ss 03:14 0:06 /usr/bin/zsh
root 1244 0.2 0.5 319036 11048 ? Ssl 03:14 0:03 /usr/libexec/upowerd
message+ 502 0.1 0.3 9976 6668 ? Ss 03:13 0:03 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog
-only
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter

```
    ps aux --sort=-%cpu | head
    }
    dysrm-info
    disk-usage
    memory-usage
    logged-users
    running-processes
```

2.$ chmod +x exp9.sh
3.$ sh exp9.sh

PROGRAM AND EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the Bash shell script using functions to automate basic administrative tasks in Linux has been done successfully.

EXP. No. 10	BASH SCRIPTING FOR ADMINISTRATIVE TASK AUTOMATION
DATE:	

AIM:

To develop a Bash shell script to perform basic file management and user account maintenance task in Linux.

ALGORITHM:

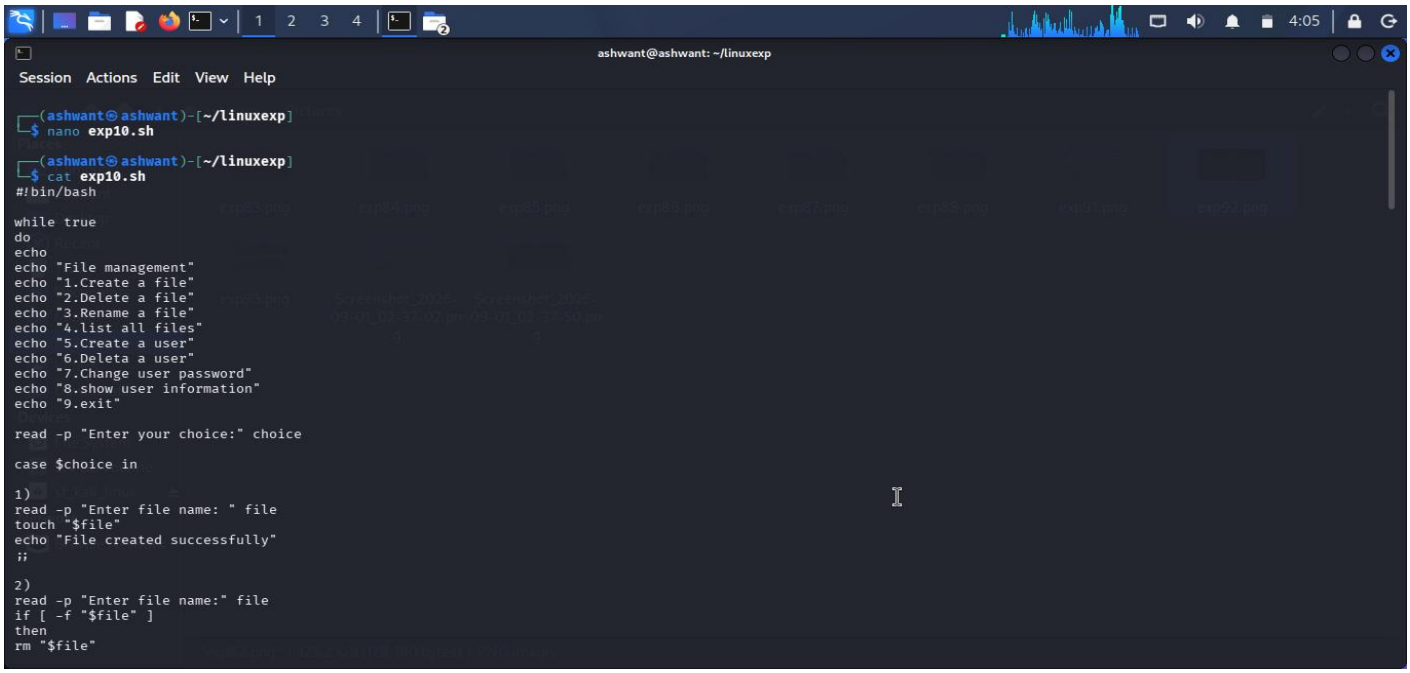
1. Write Bash script.
2. Create, delete, rename and list file.
3. Create and delete user accounts.
4. Change user password and display user information.
5. Display the results of each operation.

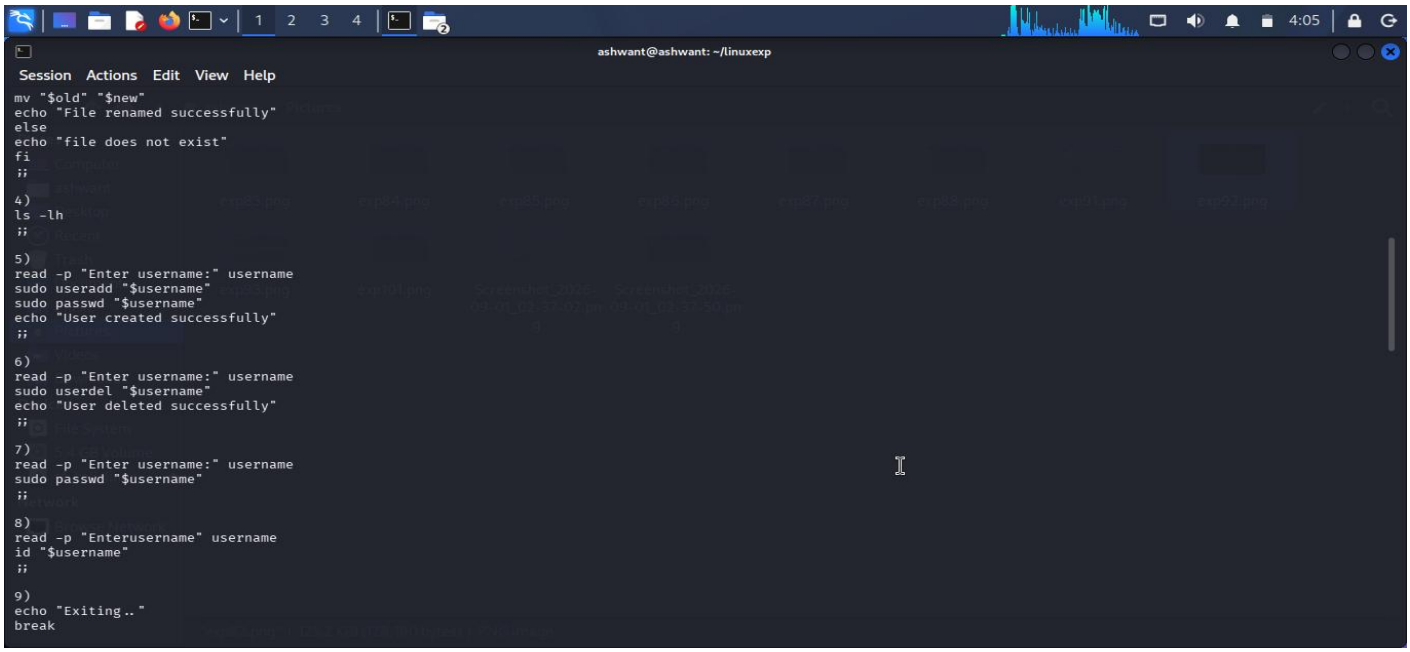
COMMANDS:

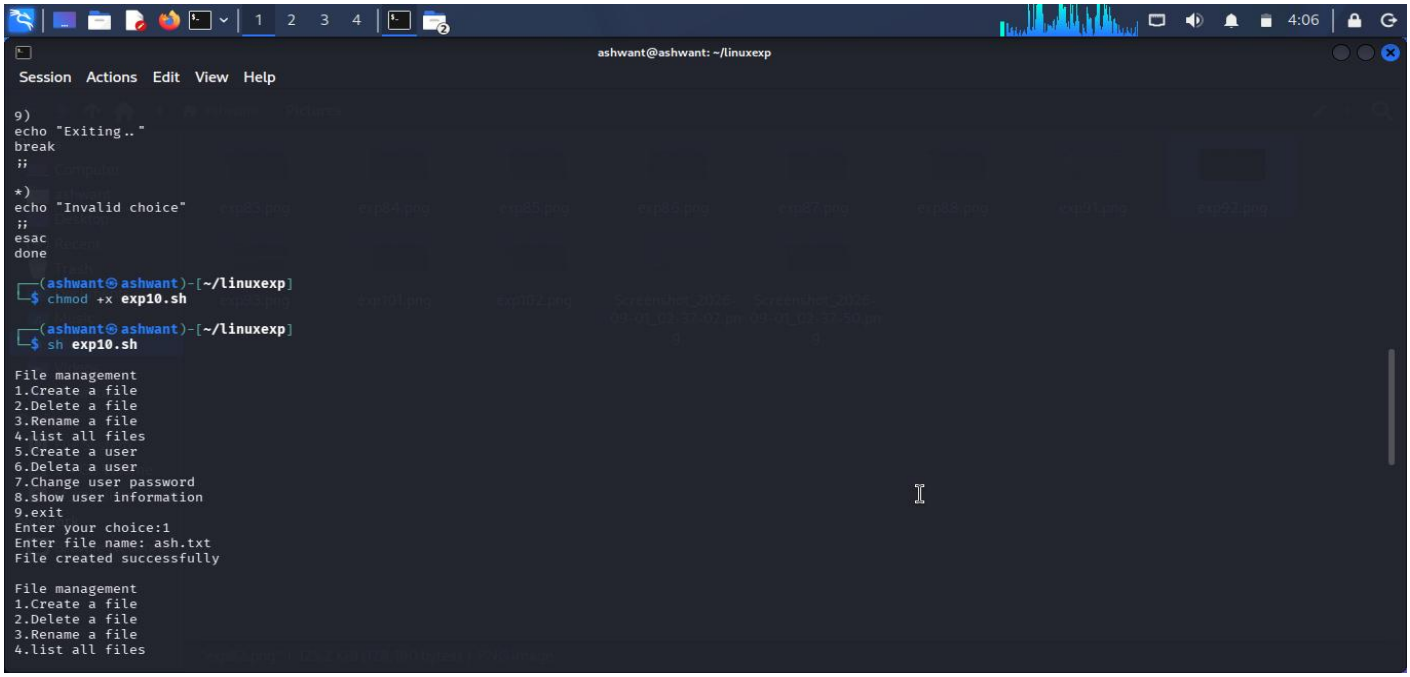
1.$ nano exp10.sh

Scripting:

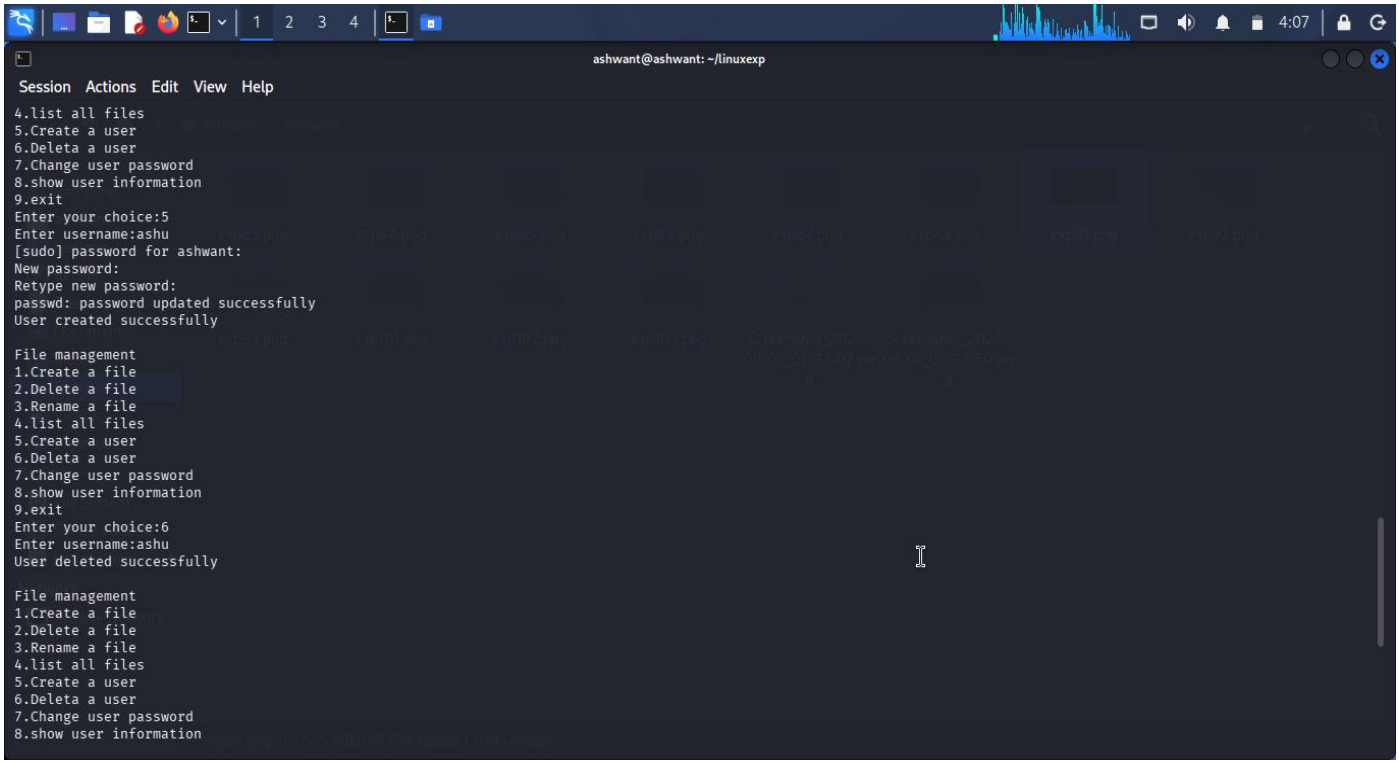
```
#!/bin/bash
while true
do
echo "File Management"
echo "1. Create a file"
echo "2. Delete a file"
echo "3. Rename a file"
echo "4. List all files"
echo "5. Create a user"
echo "6. Delete a user"
echo "7. Change user password"
echo "8. Show user information"
echo "9. Exit"
read -p "Enter your choice: " choice
case $choice in
1)
read -p "Enter file name: " file
touch "$file"
echo "File created successfully"
;;
2)
read -p "Enter file name: " file
if [ -f "$file" ]
then
rm "$file"
```

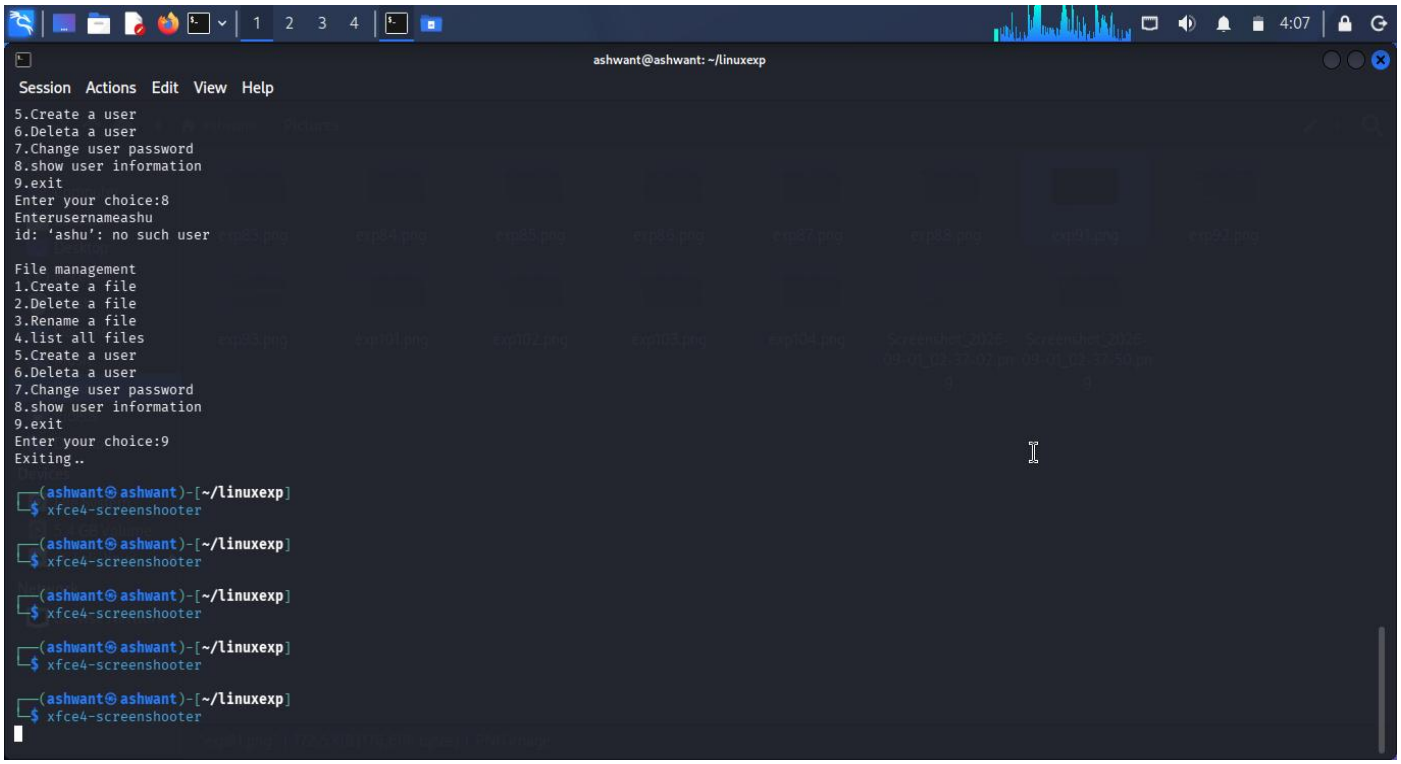

ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
(ashwant㉿ashwant)-[~/linuxexp]
$ nano exp10.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ cat exp10.sh
#!bin/bash
while true
do
echo
echo "File management"
echo "1.Create a file"
echo "2.Delete a file"
echo "3.Rename a file"
echo "4.list all files"
echo "5.Create a user"
echo "6.Deleta a user"
echo "7.Change user password"
echo "8.show user information"
echo "9.exit"
read -p "Enter your choice:" choice
case $choice in
1)
read -p "Enter file name: " file
touch "$file"
echo "File created successfully"
;;
2)
read -p "Enter file name:" file
if [-f "$file"]
then
rm "$file"


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
mv "$old" "$new"
echo "File renamed successfully"
else
echo "file does not exist"
fi
;;
4)
ls -lh
;;
5)
read -p "Enter username:" username
sudo useradd "$username"
sudo passwd "$username"
echo "User created successfully"
;;
6)
read -p "Enter username:" username
sudo userdel "$username"
echo "User deleted successfully"
;;
7)
read -p "Enter username:" username
sudo passwd "$username"
;;
8)
read -p "Enterusername" username
id "$username"
;;
9)
echo "Exiting.."
break


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
9)
echo "Exiting.."
break
;;
*)
echo "Invalid choice"
;;
esac
done
(ashwant㉿ashwant)-[~/linuxexp]
$ chmod +x exp10.sh
(ashwant㉿ashwant)-[~/linuxexp]
$ sh exp10.sh
File management
1.Create a file
2.Delete a file
3.Rename a file
4.list all files
5.Create a user
6.Deleta a user
7.Change user password
8.show user information
9.exit
Enter your choice:1
Enter file name: ash.txt
File created successfully
File management
1.Create a file
2.Delete a file
3.Rename a file
4.list all files

```
echo "File deleted successfully"
else
echo "File not exist"
fi
;;
3)
read -p "Enter old file name: " old
read -p "Enter new file name: " new
if [ -f "$old" ]
then
mv "$old" "$new"
echo "File rename successfully"
else
echo "File does not exist"
fi
;;
4)
ls -l
;;
5)
read -p "Enter username: " username
sudo useradd "$username"
echo "User created successfully"
;;
6)
read -p "Enter username: " username
sudo userdel "$username"
echo "User deleted successfully"
;;
7)
read -p "Enter username: " username
sudo passwd "$username"
;;
8)
read -p "Enter username: " username
id "$username"
;;
9)
echo "Exit"
break
;;
*)
echo "Invalid choice"
;;
esac
done
```


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
4.list all files
5.Create a user
6.Deleta a user
7.Change user password
8.show user information
9.exit
Enter your choice:5
Enter username:ashu
[sudo] password for ashwant:
New password:
Retype new password:
passwd: password updated successfully
User created successfully
File management
1.Create a file
2.Delete a file
3.Rename a file
4.list all files
5.Create a user
6.Deleta a user
7.Change user password
8.show user information
9.exit
Enter your choice:6
Enter username:ashu
User deleted successfully
File management
1.Create a file
2.Delete a file
3.Rename a file
4.list all files
5.Create a user
6.Deleta a user
7.Change user password
8.show user information


ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
5.Create a user
6.Deleta a user
7.Change user password
8.show user information
9.exit
Enter your choice:8
Enterusernameashu
id: 'ashu': no such user
File management
1.Create a file
2.Delete a file
3.Rename a file
4.list all files
5.Create a user
6.Deleta a user
7.Change user password
8.show user information
9.exit
Enter your choice:9
Exiting..
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter
(ashwant㉿ashwant)-[~/linuxexp]
$ xfce4-screenshooter

2.$ chmod +x exp10.sh
3.$ sh exp10.sh

PROGRAM AND EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the Bash shell script to perform basic file management and user account maintenance tasks in Linux has been done successfully.

.

EXP. No. 11	CREATE AND MANAGE USER,LOOPS,PASSWD AND

DATE:	ADMINISTRATIVE PRIVILEDGES USING SUDO

AIM:

To create and manage users and groups and assign Linux privileges using sudo commands.

ALGORITHM:

1. Create a new user with user privileges enabled.
2. Set the password for the user using passwd.
3. Create a group using groupadd.
4. Add the user to the group using usermod.
5. Add the user to the sudo group to provide administrative privileges.
6. Verify the new group information using id and getent.

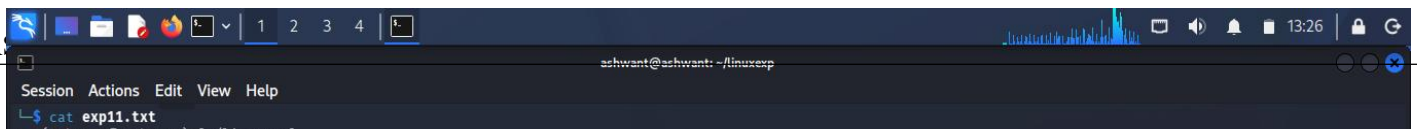
CODING:

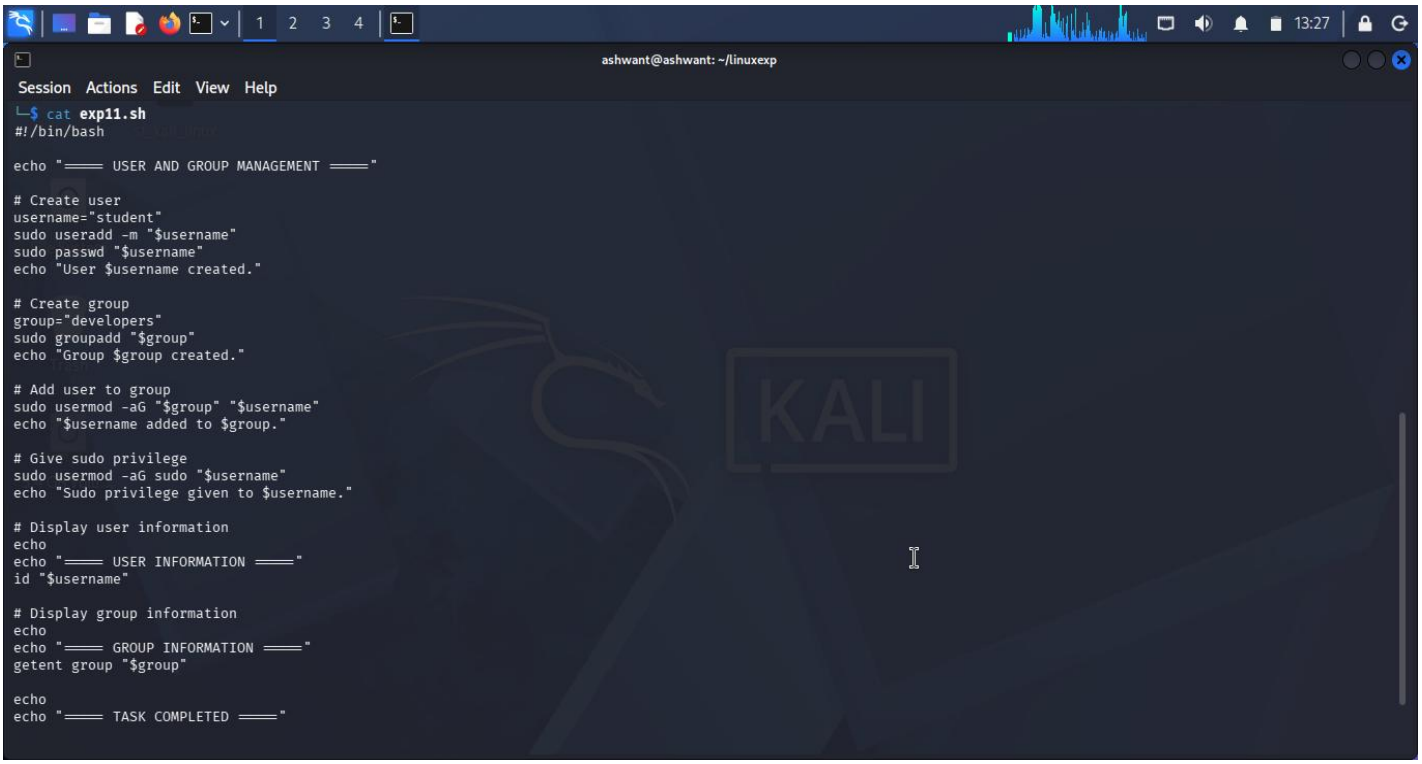
1.$nano exp11.sh

```
#Scripting
        #!/bin/bash
        echo "User and group management"
        uname = "student"
        echo "User created"
        echo "User added to group"
        group = "developers"
        sudo groupadd "group"
        sudo useradd "group"
        sudo usermod -aG group "username"
        sudo usermod -aG sudo "username"
        id "username"
        getent group "group"
```

2.$chmod +x exp11.sh

3.$sh exp11.sh




1 2 3 4
13:27
ashwant@ashwant: ~/linuxexp
Session Actions Edit View Help
└─$ cat exp11.sh
#!/bin/bash
echo "═══ USER AND GROUP MANAGEMENT ═══"
Create user
username="student"
sudo useradd -m "$username"
sudo passwd "$username"
echo "User $username created."
Create group
group="developers"
sudo groupadd "$group"
echo "Group $group created."
Add user to group
sudo usermod -aG "$group" "$username"
echo "$username added to $group."
Give sudo privilege
sudo usermod -aG sudo "$username"
echo "Sudo privilege given to $username."
Display user information
echo
echo "═══ USER INFORMATION ═══"
id "$username"
Display group information
echo
echo "═══ GROUP INFORMATION ═══"
getent group "$group"
echo
echo "═══ TASK COMPLETED ═══"

PROGRAM AND EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus to create and manage users and groups and assign Linux privileges using sudo commands done successfully

EXP. No. 12	CONFIGURE DISK PARTITION FILE SYSTEM AND

DATE:	STORAGE MANAGEMENT

AIM:

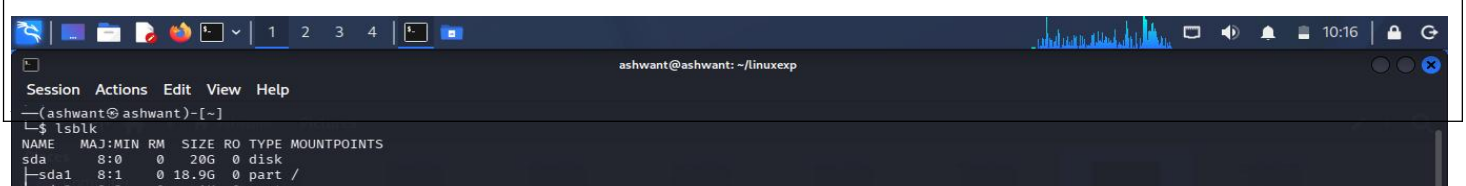
To perform disk partition, create a file system, mount the partition and manage storage in Linux.

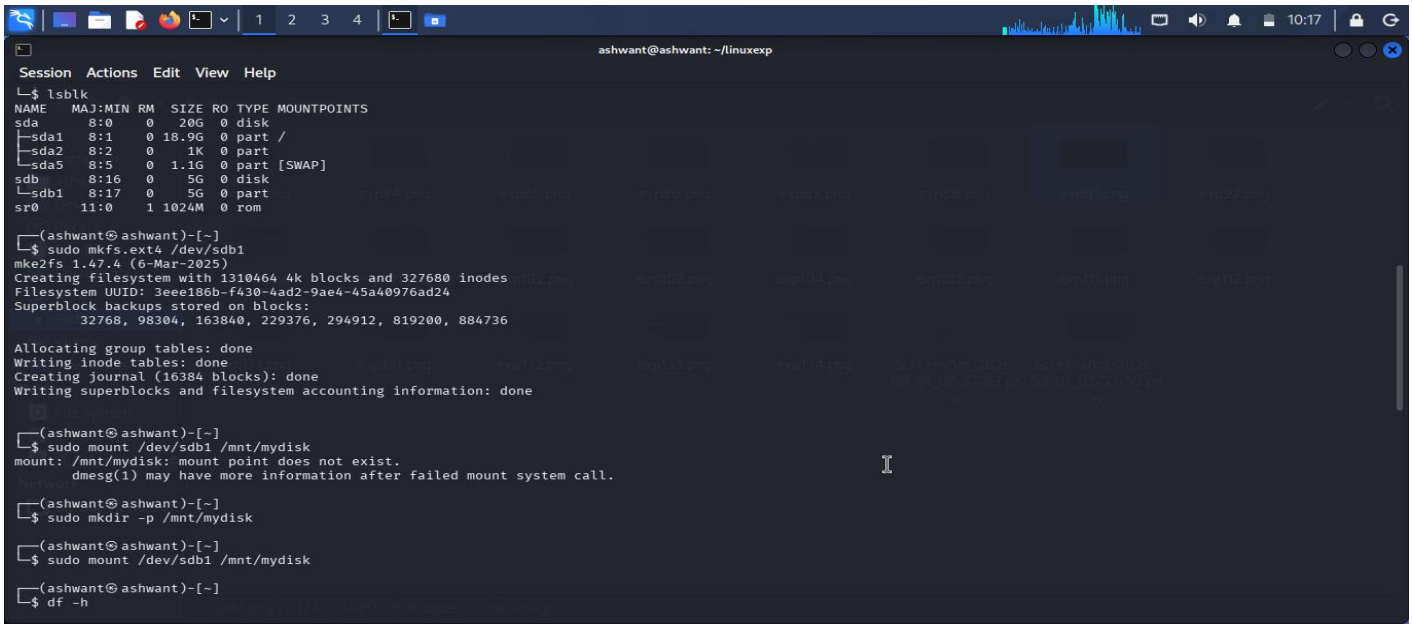
ALGORITHM:

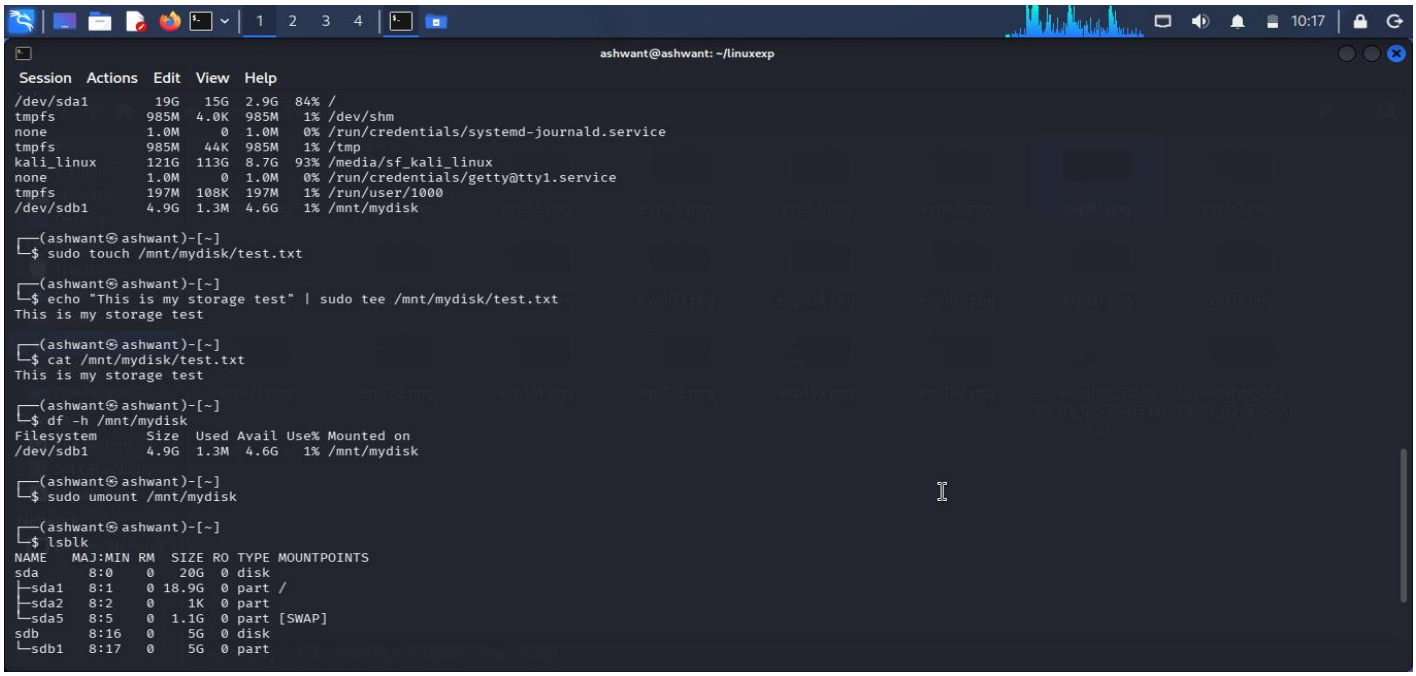
1. Identify the disk using lsblk.
2. Create a partition using fdisk.
3. Create an ext4 file system using mkfs.
4. Create a mount point and mount the partition.
5. Verify storage using df and lsblk.
6. Unmount the partition.

COMMANDS:

1. To list available disk and partition
$ lsblk
2. To open the fdisk utility to create/manage partition
$ sudo fdisk /dev/sdb
3. Create a new partition inside disk
n
4. Select primary partition type
p
5. Write the partition changes to disk and exit fdisk
W
6. To list available disk and partition
$ lsblk
7. To create an ext4 filesystem
$ sudo mkfs.ext4 /dev/sdb1
8. To mount the partition
$ sudo mount /dev/sdb1 /mnt/mydisk
9. Create the mount-point directory
$ sudo mkdir -p /mnt/mydisk
10. Show disk filesystem space usage in human readable format
$ df –h
11.Create an empty test.txt file in the mounted partition
$ sudo touch /mnt/mydisk/test.txt
12.Write text with permission
$ echo "This is my storage test" | sudo tee /mnt/mydisk/test.txt
13.To display content
$ cat /mnt/mydisk/test.txt







14. To show storage

$ df -h /mnt/mydisk

15. To unmount

$ sudo umount /mnt/mydisk

PROGRAM AND EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus to perform disk partition, create a file system, mount the partition and manage storage in Linux done successfully